

Project Report

LearnPulse — AI-Powered Student Productivity & Behavioral Analytics Dashboard

Project Type: End-to-End Data Analytics

Domain: EdTech / Educational Analytics

Tools: Python · MySQL · SQL · Power BI

Student Name: Pankaj Kumar Saini

Domain: Data Analyst

1. Problem Statement

Modern e-learning platforms generate enormous volumes of student behavioral data every day — session durations, video completions, focus patterns, login streaks, and engagement scores. Yet the majority of educational institutions and EdTech companies remain unable to leverage this data for real-time, actionable intelligence.

The result is a silent crisis: students burn out without intervention, focus declines go undetected for weeks, and struggling learners receive no early support — because reporting systems are reactive, delayed, and incapable of surfacing root causes of poor learning outcomes.

LearnPulse was designed to close this gap. Built as a full end-to-end analytics project, it transforms raw student behavioral data into an intelligent Power BI dashboard — enabling educators and administrators to identify at-risk students, understand behavioral patterns, and take evidence-based action before learners disengage entirely.

1.1 Core problems addressed:

Problem	Impact	Severity
No real-time burnout detection	Students disengage before anyone notices	Critical
No behavioral segmentation	One-size-fits-all teaching approach	High
No focus trend monitoring	Focus fatigue goes undetected	High
Streak breaks not tracked	Consistency gaps missed entirely	Medium
Efficiency not measured	High hours do not guarantee outcomes	Medium
No unified student view	Data siloed across systems	High

2. Project Objective

The objective of LearnPulse is to build a production-grade analytics pipeline that ingests, preprocesses, analyzes, and visualizes student behavioral data — delivering a single dashboard that answers the six most critical questions in student performance management:

- Who is at risk of burning out — and what are the warning signals?
- Which students are declining in focus, and how fast?
- Are students learning efficiently, or just logging hours?
- Which behavioral profile groups need targeted interventions?
- What is the true relationship between consistency and outcomes?
- How are engagement, productivity, and momentum interconnected?

To answer these questions, the project delivers: a cleaned 1,000-student dataset, 15 SQL analytical queries, Python-based feature engineering, a normalized MySQL database, and a fully interactive Power BI dashboard.

3. Business Understanding

From a business perspective, LearnPulse addresses the EdTech retention and engagement problem. Research consistently shows that students who exhibit early signals of burnout — high break frequency, declining streaks, dropping engagement — are significantly more likely to abandon courses within 30 days.

For platform operators, every dropout represents both a revenue loss and a reputational cost. For educators, it represents a teaching failure that better data could have prevented. For students, it represents a missed learning opportunity that compounds over time into persistent skill gaps.

Stakeholder	Business Value
Platform Administrators	Real-time burnout alerts, completion tracking, efficiency dashboards
Educators / Instructors	Student segmentation, focus trend visibility, intervention triggers
Data Teams	Clean analytics-ready dataset, 15 reusable SQL queries, MySQL integration
Students (indirect)	Earlier intervention, personalized learning paths, reduced dropout risk

4. Dataset Collection

The LearnPulse dataset was assembled by sourcing 2–3 related datasets from Kaggle covering student learning behaviors, engagement metrics, and academic performance indicators. The original files were in Excel (.xlsx) format and were converted to CSV using Python before the preprocessing pipeline began.

Attribute	Value
Source	Kaggle (2–3 educational behavioral datasets)
Original Format	Excel (.xlsx) — converted to CSV via Python
Final Records	1,000 students
Total Columns	18 behavioral and performance features
Student ID Format	STU_0001 to STU_1000
Profile Categories	consistent (398), struggling (351), overachiever (251)
Burnout At-Risk	481 students (48.1%)
Course Completed	639 students (63.9%)

5. Data Merging Strategy

The 2–3 source datasets from Kaggle were merged using a common student identifier field. The merging strategy was designed to produce a single analytics-ready flat table capturing every behavioral, engagement, and performance dimension required for the dashboard KPIs.

- Identified a common key (student ID) across all source datasets
- Applied an inner join strategy to retain only students present across all sources
- Resolved column naming conflicts by mapping to a standardized schema during merge
- Validated row counts before and after merging to ensure no data loss
- Resulting unified dataset: 1,000 student records with 18 analytical columns

Post-merge, the dataset was inspected for null values, duplicate rows, mismatched data types, and columns requiring renaming for SQL and Power BI compatibility

6. Data Cleaning

Data cleaning was performed entirely in Python using the Pandas library inside learnpulse.ipynb. The cleaning phase addressed four categories of quality issues.

Step 1 — Column Renaming

All 18 columns were renamed from their original source names to a standardized, analytics-compatible schema. Source datasets used inconsistent naming conventions, abbreviated labels, and whitespace-padded headers that would cause issues in MySQL and Power BI.

python

```
df.columns = [  
    'user_id', 'profile', 'daily_study_hours', 'focus_score',  
    'completed_videos', 'daily_breaks', 'monthly_active_days',  
    'streak_length', 'consistency_index', 'learning_efficiency',  
    'productivity_score', 'focus_growth', 'is_burnout_risk',  
    'course_completed', 'efficiency_ratio', 'is_break_overload',  
    'engagement_score', 'has_positive_momentum'  
]
```

Step 2 — Null Value Check

A full null audit was performed across all 18 columns. The dataset was confirmed to have zero missing values — ensuring downstream SQL aggregations and Power BI calculations would not be skewed.

python

```
print(df.isnull().sum())
```

Step 3 — Duplicate Removal

python

```
before = df.shape[0]  
df.drop_duplicates(inplace=True)  
after = df.shape[0]  
print("Duplicates Removed:", before - after)
```

Step 4 — Data Type Correction

Four binary flag columns were stored as floating-point values in the raw source data. These were cast to integer (0/1) to ensure correct GROUP BY behavior in SQL and accurate conditional formatting in Power BI.

Python

```
binary_cols = ['is_burnout_risk', 'course_completed',  
              'is_break_overload', 'has_positive_momentum']  
  
for col in binary_cols:  
    df[col] = df[col].astype(int)
```

7. Data Transformation

Outlier Detection — IQR Method

An IQR-based outlier audit was applied to every numeric column. Records outside the $1.5 \times$ IQR fence were flagged and counted. Values were retained rather than removed because in student behavior data, extremes (very high study hours, zero streaks) represent genuinely real behavioral states — removing them would distort the burnout and efficiency analyses.

python

```
for col in df.select_dtypes(include=np.number).columns:  
    Q1 = df[col].quantile(0.25)  
    Q3 = df[col].quantile(0.75)  
    IQR = Q3 - Q1  
    lower = Q1 - 1.5 * IQR  
    upper = Q3 + 1.5 * IQR  
    outliers = df[(df[col] < lower) | (df[col] > upper)]  
    print(f"{col}: {len(outliers)} outliers")
```

Profile Encoding — One-Hot Encoding

The categorical 'profile' column was one-hot encoded using `pd.get_dummies()` with `drop_first=True` to avoid multicollinearity. This produced two binary indicator columns — `profile_overachiever` and `profile_struggling` — used in SQL Query Q4 for profile-level comparisons.

python

```
df = pd.get_dummies(df, columns=['profile'], drop_first=True)  
# Creates: profile_overachiever, profile_struggling  
# Baseline: consistent (dropped to avoid multicollinearity)
```

8. Feature Engineering

Three new analytical features were engineered from existing columns to power the most important dashboard KPIs.

Feature	Formula	Business Question Answered
productivity_per_hour	$\text{productivity_score} \div \text{study_hours}$	Are students learning effectively per hour, or just logging time?
video_completion_rate	$\text{completed_videos} \div \text{active_days}$	How consistently are students engaging with content daily?
healthy_learning_pattern	$\text{burnout_risk} == 0 \text{ AND } \text{consistency_index} > 70 \rightarrow 1 \text{ else } 0$	Which students are in a genuinely sustainable learning rhythm?

```
python
# Feature 1: Productivity efficiency per hour of study
df['productivity_per_hour'] = df['productivity_score'] / df['study_hours']

# Feature 2: Daily video engagement rate
df['video_completion_rate'] = df['completed_videos'] / df['active_days']

# Feature 3: Healthy learning pattern flag
df['healthy_learning_pattern'] = np.where(
    (df['burnout_risk'] == 0) & (df['consistency_index'] > 70),
    1, 0
)
```

The healthy_learning_pattern feature became one of the core KPIs on the dashboard — showing that approximately 47% of students maintain a sustainable learning pattern, while 53% show signs of stress, inconsistency, or burnout risk.

9. Excel-to-CSV Conversion Process

The original Kaggle source files were in Excel (.xlsx) format. Before the preprocessing pipeline began, these were converted to CSV using Python's Pandas library.

Excel files introduce complications in analytics pipelines — merged cells, multiple sheets, hidden formatting, and embedded formulas can cause parsing errors in downstream tools. By converting to CSV first, the pipeline gained a clean, flat, dependency-free format compatible with all downstream tools: Python pandas, MySQL, and Power BI.

The conversion used `pd.read_excel()` followed by `df.to_csv()`, with explicit handling of sheet names and encoding parameters to prevent character set issues.

10. SQL Database Integration

After preprocessing and feature engineering, the final DataFrame was pushed to a local MySQL database using SQLAlchemy. This step transformed the flat CSV file into a queryable relational database, enabling the 15 SQL analytical queries and direct Power BI connectivity.

Using `if_exists='replace'` ensured the table was recreated cleanly on each pipeline run during development. In a production deployment, this would be replaced with `if_exists='append'` with a change-tracking strategy.

```
from sqlalchemy import create_engine

username = "root"
password = "your_password"
host     = "localhost"
port     = "3306"
database = "learnpulse"

engine = create_engine(
    f"mysql+pymysql://{username}:{password}@{host}:{port}/{database}"
)

# Write DataFrame to MySQL students table
df.to_sql("students", engine, if_exists="replace", index=False)

# Verify — read back first 5 rows
pd.read_sql("SELECT * FROM students LIMIT 5;", engine)
```

11. MySQL Table Design

The students table in the learnpulse database holds all 18 original columns plus the 3 engineered features — 21 analytical columns total. The schema was generated automatically by SQLAlchemy's `df.to_sql()` which infers MySQL data types from Pandas dtypes.

12. SQL Queries Used

15 analytical queries were written in learnpulse.sql to extract every KPI, segment, trend, and business insight surfaced on the Power BI dashboard.

13. Data Modeling for Power BI

Power BI was connected to the MySQL learnpulse database using the native MySQL connector (Get Data → MySQL Database). The students table was imported in Import mode for fast dashboard rendering.

Modeling decisions:

- Single flat table architecture — no star schema required since all dimensions are already denormalized
- burnout_risk, break_overload, positive_momentum set as Whole Number for accurate COUNT aggregations
- focus_trend, efficiency_ratio, consistency_index set as Decimal Number for average calculations
- user_id set as Text — used as drill-through identifier only, never aggregated
- profile columns used as boolean slicers in dashboard filter.

DAX Measures:

```
Burnout Students =  
CALCULATE(COUNTROWS(students), students[burnout_risk] = 1)  
  
Healthy % =  
DIVIDE(  
    CALCULATE(COUNTROWS(students), students[healthy_learning_pattern] = 1),  
    COUNTROWS(students)  
) * 100  
  
Completion Rate =  
DIVIDE(  
    CALCULATE(COUNTROWS(students), students[completion_status] = 1),  
    COUNTROWS(students)  
) * 100  
  
Avg Efficiency = AVERAGE(students[efficiency_ratio])
```

POWER BI DASHBOARD



14. Power BI Dashboard Development

The LearnPulse dashboard was built in Power BI Desktop with a structured layout organized into three functional zones: a KPI summary bar at the top, core analytical visuals in the center, and a key insight banner at the bottom.

15. KPI Design

KPI	Value	Target	Status
Total Students	1,000	—	Platform cohort
Avg Engagement Score	56.41 / 100	> 70	Below target
Avg Productivity Score	54.79 / 100	> 70	Below target
Avg Learning Efficiency	28.37	> 40	Critically low
Burnout Risk	481 (48.1%)	< 20%	Critical
Course Completion Rate	63.90%	> 70%	Below threshold
Avg Focus Score	65.92 / 100	> 75	Below target
Healthy Learning Pattern	47.10%	> 60%	Below threshold

16. Insights Generated

Burnout Crisis — 48.1% At Risk The most significant finding is that 481 out of 1,000 students are flagged at burnout risk. The burnout flag correlates strongly with high break frequency, declining engagement, and low streak lengths — confirming that burnout here is behaviorally driven, not just academically driven.

Focus Trend Analysis Students with `focus_trend < 0` are in Declining Focus — their attention has measurably worsened over the observation period. This group represents the primary early-intervention target on the dashboard.

Profile Performance Gap Overachievers (251 students) consistently outperform other profiles in productivity, focus, engagement, and consistency — despite similar average study hours. Quality of engagement matters more than quantity of time.

Diminishing Returns from Study Hours Q7 reveals a clear non-linear relationship: productivity rises with study hours up to 4–5 hours/day, then plateaus and declines for students studying 6+ hours. This confirms diminishing returns and supports capping daily sessions at 5 hours.

17. Business Recommendations

- Introduce a study-hour monitoring system that alerts students after exceeding 5 hours of continuous study time. This can help reduce burnout risk and maintain healthier learning patterns among high study-hour learners.
- Implement gamification features such as daily streak reminders, achievement badges, and consistency rewards for students with low learning streaks. This strategy can improve course completion rates and increase platform engagement.
- Deploy weekly burnout-risk alerts for educators and mentors to identify at-risk students early. Early intervention can improve retention and provide timely academic support.
- Create profile-specific learning paths for struggling learners based on performance metrics, engagement patterns, and study behavior. Personalized learning recommendations can help reduce the performance gap.
- Send engagement nudges and productivity reminders to students showing excessive break patterns or low focus consistency. This can improve concentration levels and learning efficiency.
- Organize mentoring or coaching sessions for students falling into the bottom efficiency segment. Guidance focused on time management and learning strategy optimization can improve outcomes per study hour.

18. Challenges Faced & Solutions

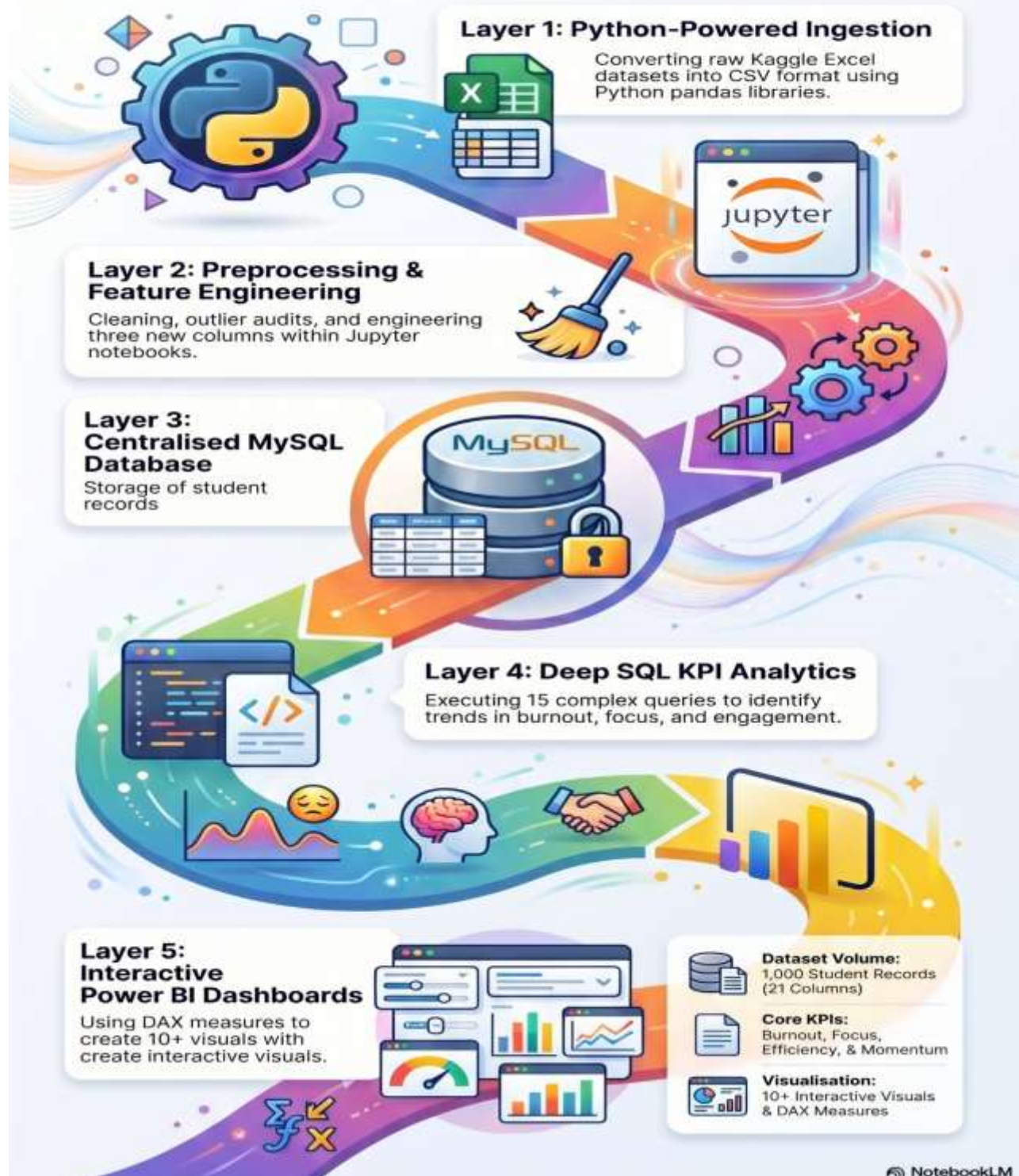
- Multiple Kaggle datasets contained inconsistent column names and formatting structures. To solve this, all column names were standardized before preprocessing and integration to ensure consistency across the pipeline.
- Several binary flag columns were stored as float values, which created issues during SQL aggregation and GROUP BY operations. These columns were converted into integer format during the cleaning phase using Python.
- Outliers in study-hour related fields significantly affected KPI averages and dashboard insights. Outlier handling techniques and threshold-based filtering were applied to maintain analytical accuracy.
- Merging multiple datasets introduced duplicate records and missing values. Data cleaning and validation steps were performed carefully to create a reliable analytics-ready dataset.
- Establishing smooth integration between Python, MySQL, and Power BI required proper datatype alignment and query optimization For Dashboard.

- Predictive dropout model using scikit-learn — logistic regression or random forest trained on burnout_risk as target variable
- Real-time data pipeline using Apache Kafka or scheduled Python scripts for live data ingestion
- Power BI Service deployment for browser-based sharing without Power BI Desktop requirement
- Multi-table normalized MySQL schema — separate tables for students, activity_log, and performance_metrics
- Integration with live LMS APIs (Moodle, Canvas, Coursera) replacing synthetic Kaggle data
- Automated weekly email alerts to educators when students cross burnout or consistency thresholds
- Student self-service dashboard — a simplified personal metrics view accessible directly to students.

19. Tech Stack Used

Layer	Tool / Library	Version	Purpose
Language	Python	3.10	Data processing, feature engineering, DB push
Data Manipulation	Pandas	2.1.0	DataFrame operations, cleaning, transformation
Numerical	NumPy	1.25	Array operations, feature engineering logic
Database	MySQL	8.0	Relational storage for analytics-ready dataset
ORM / Connector	SQLAlchemy + PyMySQL	2.0 / 1.1	Python-to-MySQL connectivity
IDE	Jupyter Notebook	—	Interactive preprocessing and development
BI Tool	Power BI Desktop	Latest	Dashboard, KPIs, DAX measures
SQL	MySQL SQL	—	15 analytical queries, segmentation, KPIs
Data Source	Kaggle	—	2–3 educational behavioral datasets (Excel)

The LearnPulse Data Engineering & Analytics Pipeline



21. Conclusion

LearnPulse demonstrates that a complete, production-quality analytics solution can be built from publicly available data using a focused, well-architected pipeline. Starting from 2–3 raw Kaggle Excel files and ending with a live interactive Power BI dashboard, the project covers every stage of the modern data analytics workflow — collection, merging, cleaning, transformation, feature engineering, database integration, SQL analytics, data modeling, and dashboard deployment.

The findings are both analytically significant and business-relevant. Nearly half the student cohort is at burnout risk. Average productivity and focus scores fall below their target thresholds. Course completion stands at 63.9%. These are not abstract numbers — they are the kinds of signals that, in a real platform, would trigger immediate action from product teams, educators, and data teams.

The project showcases the technical depth of Python data engineering, advanced SQL analytics, and Power BI development — while staying grounded in the business problem it was designed to solve. It is a portfolio piece that reflects both analytical capability and real-world problem-solving judgment.

"Without data, you are just another person with an opinion."

LearnPulse turns student data into decisions -- and decisions into outcomes.